

Airbnb Hotel Booking Analysis

(NYC)

Data Analysis — Full Project Documentation

Python · MySQL · Power BI Desktop

96,698 Listings · \$625.85 Avg Price · 3.29 / 5 Avg Rating · 5 NYC Boroughs

TOOLS	SOURCE	RECORDS	DATE RANGE
Python · MySQL · Power BI	NYC Airbnb Open Data	96,698 · 24 Cols	2012 – 2022 built

01 EXECUTIVE SUMMARY

- **Cleaned and analyzed** 102,599 raw NYC Airbnb listings across 26 columns down to a 96,698-row, 24-column analysis-ready dataset — removing 541 exact duplicates, 2 unusable columns, misspelled borough labels, and 4,001 rows of impossible outlier values (availability outside 0–365 days, negative minimum-night stays) through a fully scripted, repeatable Python pipeline.
- **Answered 9 structured business questions** covering listing distribution, pricing, construction year, host behavior, and review patterns — then reproduced every answer independently in MySQL using single-table SQL (including a hand-built Pearson correlation formula, since MySQL has no native CORR() aggregate) to cross-validate the Python results to 3+ decimal places.
- **Built a 2-page, 15-visual interactive Power BI dashboard** with 6 DAX measures, 2 KPI-card rows, 5 bar charts, 1 conditionally-formatted matrix heatmap, 2 trend-lined scatter plots, and 3 cross-filtering slicers — connected directly to the MySQL database rather than a static file export.
- **Surfaced a genuinely counter-intuitive finding:** average price is essentially flat across all five boroughs (\$623.75–\$631.34, a \$7.59 spread), and price shows no meaningful relationship with construction year ($r = -0.005$) or host verification status — evidence that price in this dataset is closer to randomly assigned than market-driven, which is reported as a data-quality caveat rather than a real pricing insight.

02 PROBLEM STATEMENT

Airbnb's New York City listings data was distributed as a single flat export with well-known, undocumented quality issues: duplicate records, two columns (house_rules, license) that were almost entirely empty, inconsistent spelling of the neighbourhood_group field ('brookln', 'manhatan'), unlabeled currency symbols in price fields, and physically

impossible values in `availability_365` and `minimum_nights`. Without a documented, reproducible cleaning process, any analysis built on top of the raw file risked silently double-counting listings, miscoding boroughs, or treating data-entry errors as real signal. The goal of this project was to build that reproducible pipeline once, then answer 9 concrete questions about the market — property types, borough pricing, host behavior, and review patterns — through three complementary lenses: Python for exploratory analysis, SQL for validated relational querying, and Power BI for an interactive, presentable dashboard.

03 OBJECTIVES

- **Obj 1:** Diagnose and document every data-quality issue in the raw dataset before writing any cleaning code, so each fix is deliberate and traceable rather than a blind blanket transformation.
- **Obj 2:** Clean the dataset in a scripted, repeatable Python pipeline (pandas) and answer all 9 project questions with summary statistics and matplotlib/seaborn visualizations.
- **Obj 3:** Load the cleaned data into a MySQL database and reproduce every Python finding as an independent SQL query, validating both approaches agree.
- **Obj 4:** Build DAX measures and a 2-page interactive Power BI dashboard connected live to MySQL, enabling cross-filtering by borough and room type.
- **Obj 5:** Surface actionable, correctly-scaled insights — and just as importantly, correctly flag where the data shows **no** relationship, rather than overstating noise as a finding.
- **Obj 6:** Produce a portfolio-ready deliverable demonstrating the full pipeline: raw file → Python EDA → relational SQL → BI dashboard, with one consistent source of truth throughout.

04 DATASET OVERVIEW

Source & Structure

Attribute	Detail
Dataset Name	NYC Airbnb Open Data
File Format	Microsoft Excel Workbook (.xlsx), single sheet
Raw Records	102,599 rows × 26 columns
Clean Records	96,698 rows × 24 columns (after full cleaning pipeline)
Geographic Scope	5 NYC boroughs · 224 unique neighbourhoods
Duplicate Rows Removed	541 exact duplicates
Columns Dropped	house_rules (54,843 missing), license (102,597 / 102,599 missing)
Outlier Rows Removed	4,001 total (3,633 invalid availability 365, 368 negative minimum nights)
Rows Dropped — Missing Key Values	1,359 (only where a value needed for the 9 questions was missing)

Key Columns Used in Analysis

Column	Type	Description
room_type	Text	4 categories: Entire home/apt, Private room, Shared room, Hotel room
neighbourhood_group	Text	5 NYC boroughs (misspellings corrected during cleaning)

Column	Type	Description
price	Decimal	Nightly listing price in USD (\$50 – \$1,200, avg \$625.85)
service_fee	Decimal	Platform service fee in USD (\$10 – \$240, avg \$125.17)
construction_year	Integer	Building construction year (2003 – 2022)
host_identity_verified	Text	'verified' or 'unconfirmed' (49.98% / 50.02% split)
calculated_host_listings_count	Integer	Number of listings managed by the same host (avg 8.06)
review_rate_number	Integer	Star rating, 1–5 (avg 3.29)
availability_365	Integer	Days available per year, 0–365 (avg 134.38)

05 TOOLS & TECHNOLOGIES USED

Tool	Platform	Purpose in This Project
Python 3	VS Code	Data cleaning, EDA, statistics, matplotlib/seaborn visualization
pandas / numpy	Python library	Duplicate removal, type coercion, outlier filtering, groupby/pivot analysis
MySQL 8	MySQL Workbench	Relational storage of the cleaned dataset; SQL validation of all 9 answers
Power BI Desktop	Microsoft	Interactive 2-page dashboard, DAX measures, cross-filtering slicers
Power Query	Built into Power BI	Host Label engineering (host_name + host_id) for unique chart labels
DAX	Power BI native language	6 measures: Total Listings, Avg Price, Avg Service Fee, Avg Review Rate, Avg Availability, % Verified Hosts

06 DATA CLEANING PROCESS — PYTHON

All 9 cleaning steps were executed in a single scripted pandas pipeline, run top to bottom, with row counts printed after every destructive step so each removal is auditable.

Step	Action	Result
1	Load raw .xlsx into DataFrame	102,599 rows × 26 columns
2	Drop exact duplicate rows	-541 rows → 102,058 remain
3	Drop house_rules and license columns	26 → 24 columns (near-total missingness)
4	Strip \$ and , from price / service_fee, cast to float	Numeric types restored
5	Fix neighbourhood_group misspellings ('brookln'→Brooklyn, 'manhatan'→Manhattan)	Clean 5-borough category set
6	Remove availability_365 outside 0–365 range	-3,633 rows
7	Remove negative minimum_nights	-368 rows

Step	Action	Result
8	Correct dtypes: id/host_id → Int64, Construction year → Int64, last_review → datetime	Type-safe columns
9	Drop rows missing values in the 9 analysis-critical columns only	-1,359 rows → 96,698 final

07 SQL / MYSQL — VALIDATION LAYER

Unlike a multi-entity schema, this dataset is a single flat table (listings), so the SQL work centers on query design and independent validation rather than relational joins. All 9 Python answers were reproduced in MySQL and matched to 3+ decimal places.

Schema & Import

- Single table listings (24 columns), imported via LOAD DATA LOCAL INFILE with a defensive TRIM(TRAILING '\r' ...) on the last column to strip Windows line-ending artifacts.
- 4 indexes added on neighbourhood_group, room_type, host_id, and host_identity_verified — the columns used in every GROUP BY.
- 1 analytical VIEW (vw_airbnb_summary) pre-aggregating listings into price/availability buckets by borough, room type, and booking type, for fast reporting without rescanning all 96,698 rows.

The Correlation Problem

MySQL has no built-in CORR() aggregate function (unlike PostgreSQL/Oracle). All 3 correlation questions (Q4, Q7, Q9) were implemented manually using the Pearson formula directly in SQL: $r = (n\sum xy - \sum x \sum y) / \sqrt{[(n\sum x^2 - (\sum x)^2)(n\sum y^2 - (\sum y)^2)]}$. Results matched pandas' built-in .corr() to within rounding: construction year vs price -0.0047 (both), price vs service fee 0.99999 in SQL vs 1.0000 in Python (the tiny gap is DECIMAL(8,2) column rounding, not a real discrepancy), and host listings count vs availability 0.1755 (both).

08 DAX MEASURES CREATED

Measure	DAX Formula	Result
Total Listings	COUNTROWS(listings)	96,698
Avg Price	AVERAGE(listings[price])	\$625.85
Avg Service Fee	AVERAGE(listings[service_fee])	\$125.17
Avg Review Rate	AVERAGE(listings[review_rate_number])	3.29 / 5
Avg Availability (days)	AVERAGE(listings[availability_365])	134.38 days
% Verified Hosts	DIVIDE(CALCULATE([Total Listings], host_identity_verified="verified"), [Total Listings])	49.98%

09 DASHBOARD VISUALS BUILT (15 Visuals, 2 Pages)

Page 1 — Market Overview

#	Visual Type	Fields Used	Insight Delivered
1-4	KPI Cards	Total Listings / Avg Price / Avg Review Rate / % Verified Hosts	96,698 listings, \$625.85 avg price, 3.29 avg rating, 49.98% verified
5	Bar Chart	Y: room_type / X: Total Listings	Entire home/apt leads at 50,543 (52.3%)
6	Bar Chart	Y: neighbourhood_group / X: Total Listings	Manhattan leads at 41,076 (42.5%)
7	Bar Chart	Y: neighbourhood_group / X: Avg Price	Prices nearly flat across boroughs (\$624–\$631)
8-9	Slicers	room_type, neighbourhood_group	Cross-filters all Page 1 visuals

Page 2 — Hosts & Reviews

#	Visual Type	Fields Used	Insight Delivered
1	Bar Chart	Y: Host Label / X: calculated_host_listings_count	40 hosts tied at 332 listings, all 'Blueground'
2	Clustered Column	X: host_identity_verified / Y: Avg Review Rate	Verified 3.29 vs Unconfirmed 3.28 — negligible gap
3	Matrix (heatmap)	Rows: neighbourhood_group / Cols: room_type / Values: Avg Review Rate	Hotel rooms in Brooklyn (3.86) and Queens (3.75) stand out
4	Scatter + trend line	X: price / Y: service_fee	Near-perfect straight line, $r \approx 1.00$
5	Scatter + trend line	X: calculated_host_listings_count / Y: availability_365	Weak upward trend, $r \approx 0.18$
6	Slicer	neighbourhood_group	Cross-filters all Page 2 visuals

10 KEY INSIGHTS & RESULTS

A. Listing Distribution

- **Entire home/apt dominates the market** at 50,543 listings (52.3%), followed by Private room at 43,938 (45.4%); Shared room and Hotel room together make up under 2.3% of supply.
- **Manhattan and Brooklyn together hold 83.3% of all listings** (41,076 and 39,445 respectively), while Staten Island accounts for under 1% (911 listings) — a roughly 45x gap between the largest and smallest borough.

B. Pricing Patterns

- **Average price is essentially flat across boroughs:** Bronx (\$631.34), Queens (\$629.51), Brooklyn (\$626.54), Staten Island (\$624.26), and Manhattan (\$623.75) — a spread of just \$7.59, which does not match the real NYC market where Manhattan commands a clear premium. This points to price being randomly generated in this teaching dataset rather than location-driven.
- **Construction year has no relationship with price** ($r = -0.0047$) — newer buildings do not command higher nightly rates in this data.
- **Price and service fee are perfectly correlated** ($r = 1.00$) — service fee is a derived field computed directly from price, not an independent signal.

C. Host Behavior

- **Corporate hosts dominate the top of the market:** the top-ranked hosts by listing count are all "Blueground," a mid-term corporate rental operator — 40 separate host accounts tied at exactly 332 listings each, distinct from the typical individual host who manages a single unit (96,697 unique hosts across 96,698 listings).
- **Host identity verification barely affects reviews:** verified hosts average 3.293 stars vs. 3.283 for unconfirmed hosts ($n = 48,325$ vs $48,373$) — a 0.01-point gap, not a meaningful difference.
- **More listings per host correlates weakly with higher availability** ($r = 0.1755$) — professional/multi-property hosts keep marginally more inventory open year-round, but the relationship is weak, not strong.

D. Review Patterns

- **Overall average review rate is 3.29 / 5** and is remarkably stable across both borough and room type (mostly 3.24–3.37).
- **Hotel room listings are the one standout segment:** Brooklyn (3.86) and Queens (3.75) hotel rooms score meaningfully higher than any other borough/room-type combination — though hotel rooms are a very small slice of the data (111 listings total), so this should be treated as a directional signal, not a robust trend.

11 CHALLENGES FACED & HOW OVERCOME

Challenge 1 — Silent Data Quality Issues

Problem: Misspelled borough labels ('brookln', 'manhatan') and impossible values (availability_365 up to 3,677, minimum_nights as low as -1,223) would have silently corrupted every downstream groupby if cleaned blindly.

Solution: Ran a dedicated assessment pass before any cleaning — checking `.unique()` on categorical columns and `.min()/max()` on numeric columns — so every fix in the pipeline targeted a specifically diagnosed problem rather than guessing.

Challenge 2 — MySQL Has No Native Correlation Function

Problem: All 3 correlation questions (construction year vs price, price vs service fee, host listings count vs availability) have no direct SQL equivalent to pandas' `.corr()` in MySQL.

Solution: Implemented the Pearson correlation formula manually using `SUM()`, `POWER()`, and `SQRT()` — validated the SQL output against the Python result for all 3 pairs, confirming agreement to 3+ decimal places.

Challenge 3 — Duplicate Host Display Names on Charts

Problem: The Top 10 Hosts chart showed 10 bars all labeled identically "Blueground," with no way to distinguish individual host accounts.

Solution: Engineered a Host Label custom column in Power Query (`host_name & " (" & host_id & ")"`), giving each bar a unique, readable label.

Challenge 4 — Tied Values Breaking a Clean Top-N Filter

Problem: Applying a standard Top N = 10 filter on host listings count didn't cleanly return 10 rows, because 40 distinct hosts are genuinely tied at the maximum value (332 listings).

Solution: Documented the tie as a real data characteristic in the chart title rather than forcing an artificial, misleading cutoff to exactly 10.

Challenge 5 — Scatter Charts Collapsing to a Single Point

Problem: The price-vs-service-fee and host-count-vs-availability scatter plots initially rendered as a single averaged dot instead of one point per listing.

Solution: Set both the X-axis and Y-axis field aggregation to "Don't summarize," restoring full row-level granularity (~96,698 individual points).

Challenge 6 — Ratio Displayed as a Raw Decimal

Problem: The % Verified Hosts KPI card showed "0.50" instead of "49.98%" because the underlying `DIVIDE()` measure returns a 0–1 ratio, and the card's field format defaulted to a plain number.

Solution: Explicitly set the card's number format to Percentage — a reminder that DAX ratio measures need their display format set independently of the calculation logic.

Challenge 7 — Chart Type Distorting a Near-Tied Comparison

Problem: A donut/pie chart comparing verified vs. unconfirmed review rates visually rendered the two values as an almost-exact 50/50 split, overstating a genuinely tiny 0.01-point gap (3.28 vs 3.29).

Solution: Switched to a clustered column chart, which preserves the true numeric scale instead of forcing two independent averages into a proportional whole.

12 SKILLS LEARNED & TAKEAWAYS

- End-to-end analytics pipeline design: raw file → Python cleaning/EDA → relational SQL storage and validation → interactive BI dashboard, with one consistent, cross-checked source of truth across all three layers.
- Systematic data-quality auditing (duplicates, category spelling, numeric range validation, dtype checks) performed before cleaning, not interleaved with it — every removal in the pipeline is traceable to a diagnosed cause.
- Translating pandas-style `groupby/pivot_table` logic into equivalent SQL — conditional aggregation (`CASE WHEN`) standing in for `pivot`, and a hand-built Pearson formula standing in for a missing built-in correlation function.
- DAX fundamentals: `DIVIDE()` for safe division, `CALCULATE()` for filtered aggregation, and the distinction between a measure's underlying value and its display format.
- Chart-type judgement: recognizing when a visualization choice (pie chart for two near-equal values) actively misrepresents the data, and knowing which alternative preserves the true scale.
- Cross-tool validation as a habit: treating agreement between Python and SQL results as a correctness check, not a redundant step.

13 CONCLUSION

- **Processed 102,599 raw NYC Airbnb records** through a 9-step, fully scripted Python cleaning pipeline — removing 541 duplicates, 2 unusable columns, and 4,001 outlier rows — reaching a 96,698-row, 24-column analysis-ready dataset with zero blind cleaning decisions.
- **Independently validated all 9 project questions in MySQL**, including a manually-implemented Pearson correlation formula for the 3 questions with no native SQL equivalent, confirming agreement with the Python results to 3+ decimal places.

- **Delivered a 2-page, 15-visual Power BI dashboard** connected live to the MySQL database, with 6 DAX measures, 3 cross-filtering slicers, and one purpose-built analytical view — surfacing that listing type and borough drive volume, but not price; that price and service fee are fully derived from one another; and that corporate hosts (not individuals) dominate the top of the host-listings ranking.
- **Most notably, correctly identified where the data shows no signal:** flat pricing across boroughs, no construction-year effect, and a negligible verification-to-review-rate relationship — treating the absence of a pattern as a real, reportable finding rather than searching for one that isn't there.

KPI Summary

Metric	Value	Metric	Value
Total Listings	96,698	Boroughs Covered	5
Avg Price	\$625.85	Avg Service Fee	\$125.17
Avg Review Rate	3.29 / 5	% Verified Hosts	49.98%
Price ↔ Service Fee Corr.	1.00	Construction Yr ↔ Price Corr.	-0.005
Host Count ↔ Availability Corr.	0.18	Top Room Type	Entire home/apt (52.3%)
Top Borough (Volume)	Manhattan — 41,076	Top Borough (Avg Price)	Bronx — \$631.34
Unique Hosts	96,697	Tied Top Hosts	40 @ 332 listings (Blueground)
Duplicates Removed	541	Total Rows Dropped (Cleaning)	5,901